

Week 1

1. Java Lambda Expressions:

Program:

```
import java.io.*;
import java.util.*;
interface PerformOperation {
    boolean check(int a);
}
class MyMath {
    public static boolean checker(PerformOperation p, int num) {
        return p.check(num);
    }

    // Write your code here
    public PerformOperation isOdd() {
        return n -> n % 2 != 0;
    }

    public PerformOperation isPrime() {
        return n -> {
            if (n < 2) return false;
            for (int i = 2; i <= Math.sqrt(n); i++) {
                if (n % i == 0) return false;
            }
            return true;
        };
    }

    public PerformOperation isPalindrome() {
        return n -> {
            int original = n;
            int reversed = 0;
            while (n > 0) {
                reversed = reversed * 10 + (n % 10);
                n /= 10;
            }
            return original == reversed;
        };
    }
}
```

```

    };
}
}
public class Solution {

    public static void main(String[] args) throws IOException {
        MyMath ob = new MyMath();
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int T = Integer.parseInt(br.readLine());
        PerformOperation op;
        boolean ret = false;
        String ans = null;
        while (T--> 0) {
            String s = br.readLine().trim();
            StringTokenizer st = new StringTokenizer(s);
            int ch = Integer.parseInt(st.nextToken());
            int num = Integer.parseInt(st.nextToken());
            if (ch == 1) {
                op = ob.isOdd();
                ret = ob.checker(op, num);
                ans = (ret) ? "ODD" : "EVEN";
            } else if (ch == 2) {
                op = ob.isPrime();
                ret = ob.checker(op, num);
                ans = (ret) ? "PRIME" : "COMPOSITE";
            } else if (ch == 3) {
                op = ob.isPalindrome();
                ret = ob.checker(op, num);
                ans = (ret) ? "PALINDROME" : "NOT PALINDROME";

            }
            System.out.println(ans);
        }
    }
}

```

Output:

Compiler Message

Success

Input (stdin)

```
5
1 4
2 5
3 898
1 3
2 12
```

Expected Output

```
EVEN
PRIME
PALINDROME
ODD
COMPOSITE
```

2. Java Comparator:

```
import java.util.*;
```

```
class Checker implements Comparator<Player> {
```

```
    @Override
```

```
    public int compare(Player p1, Player p2) {
```

```
        if (p1.score != p2.score) {
```

```
            return p2.score - p1.score;
```

```
        }
```

```
        return p1.name.compareTo(p2.name);
```

```
    }
```

```
}
```

```
class Player {
```

```
    String name;
```

```
    int score;
```

```
Player(String name, int score) {  
    this.name = name;  
    this.score = score;  
}  
}
```

```
public class Solution {  
  
    public static void main(String[] args) {  
        Scanner scan = new Scanner(System.in);  
  
        int n = scan.nextInt();  
  
        Player[] player = new Player[n];  
        Checker checker = new Checker();  
  
        for (int i = 0; i < n; i++) {  
            player[i] = new Player(scan.next(), scan.nextInt());  
        }  
  
        scan.close();  
  
        Arrays.sort(player, checker);  
  
        for (int i = 0; i < player.length; i++) {  
            System.out.println(player[i].name + " " + player[i].score);  
        }  
    }  
}
```

```
    }  
    }  
}
```

Output:

```
5  
amy 100  
david 100  
heraldo 50  
aakansha 75  
aleksa 150  
  
aleksa 150  
amy 100  
david 100  
aakansha 75  
heraldo 50
```

3. Java Sort:

Program:

```
import java.util.*;  
  
class Student {  
    private int id;  
    private String fname;  
    private double cgpa;  
  
    public Student(int id, String fname, double cgpa) {  
        this.id = id;  
        this.fname = fname;  
        this.cgpa = cgpa;  
    }  
}
```

```
public int getId() {  
    return id;  
}
```

```
public String getFname() {  
    return fname;  
}
```

```
public double getCgpa() {  
    return cgpa;  
}  
}
```

```
public class Solution {
```

```
    public static void main(String[] args) {  
        Scanner in = new Scanner(System.in);
```

```
        int testCases = in.nextInt();
```

```
        List<Student> studentList = new ArrayList<Student>();
```

```
        while (testCases-- > 0) {
```

```
            int id = in.nextInt();
```

```
            String fname = in.next();
```

```
            double cgpa = in.nextDouble();
```

```

        studentList.add(new Student(id, fname, cgpa));
    }

    Collections.sort(studentList, new Comparator<Student>() {
        @Override
        public int compare(Student s1, Student s2) {

            if (s1.getCgpa() != s2.getCgpa()) {
                return Double.compare(s2.getCgpa(), s1.getCgpa());
            }

            int nameCompare = s1.getFname().compareTo(s2.getFname());

            if (nameCompare != 0) {
                return nameCompare;
            }

            return Integer.compare(s1.getId(), s2.getId());
        }
    });

    for (Student st : studentList) {
        System.out.println(st.getFname());
    }

    in.close();

```

```
    }  
}
```

Output:

Hardik

Rumpel

Sushil

Harry

Titu

4. Sorting Comparator:

Program:

```
import java.util.*;
```

```
class Player {
```

```
    String name;
```

```
    int score;
```

```
    Player(String name, int score) {
```

```
        this.name = name;
```

```
        this.score = score;
```

```
    }
```

```
}
```

```
class Checker implements Comparator<Player> {
```

```
    public int compare(Player a, Player b) {
```



```

        if (a.score != b.score) {
            return b.score - a.score;
        }

        return a.name.compareTo(b.name);
    }
}

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();

        Player[] player = new Player[n];
        Checker checker = new Checker();

        for (int i = 0; i < n; i++) {
            player[i] = new Player(scan.next(), scan.nextInt());
        }

        scan.close();

        Arrays.sort(player, checker);

        for (int i = 0; i < player.length; i++) {
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}

```

```

    }
}
}

```

Output:

Compiler Message

Success

Input (stdin)

5

amy 100

david 100

heraldo 50

aakansha 75

aleksa 150

Expected Output

aleksa 150

amy 100

david 100

aakansha 75

heraldo 50

5. Running sum of 1D Array :

Program:

```

class Solution {
    public int[] runningSum(int[] nums) {
        // Calculate the running sum
        for (int i = 1; i < nums.length; i++) {
            nums[i] = nums[i] + nums[i-1];
        }
        return nums;
    }
}

```

Output:

Input

nums =

[1,2,3,4]

Output

[1,3,6,10]

Expected
[1,3,6,10]

6. Richest Customer Wealth:

Program:

```
class Solution {
public:
    int maximumWealth(int[][] accounts) {
        int maxWealth = 0;
        for (int i = 0; i < accounts.length; i++) {
            int wealth = 0;
            for (int j = 0; j < accounts[i].length; j++) {
                wealth += accounts[i][j];
            }
            if (wealth > maxWealth) {
                maxWealth = wealth;
            }
        }
        return maxWealth;
    }
}
```

Output:

Input

```
accounts =
[[1,2,3],[3,2,1]]
```

Output

6

Expected

6

7. Squares Of A Sorted Array:

Program:

```

import java.util.*;
public class Solution {
    public static int[] sortedSquares(int[] nums) {
        int n=nums.length;
        int[] ans=new int[n];
        int left=0;
        int right=n-1;
        int pos=n-1;
        while(left<=right){
            int leftSquare=nums[left]*nums[left];
            int rightSquare=nums[right]*nums[right];
            if(leftSquare > rightSquare){
                ans[pos]=leftSquare;
                left++;
            }
            else{
                ans[pos]=rightSquare;
                right--;
            }
            pos--;
        }
        return ans;
    }
    public static void main(String[] args){
        int[] nums={-4,-1,0,3,10};
        int[] result=sortedSquares(nums);
        System.out.println(Arrays.toString(result));
    }
}

```

Output:

Input	nums = [-4,-1,0,3,10]
Output	[0,1,9,16,100]
Expected	[0,1,9,16,100]

8. SFind Pivot Index:

Program:

```

class Solution {
    public int pivotIndex(int[] nums) {

```

```

    int totalSum = 0;

    for (int num : nums) {
        totalSum += num;
    }

    int leftSum = 0;

    for (int i = 0; i < nums.length; i++) {

        int rightSum = totalSum - leftSum - nums[i];

        if (leftSum == rightSum) {
            return i;
        }

        leftSum += nums[i];
    }

    return -1;
}

```

Output:

Input

```

nums =
[1,7,3,6,5,6]

```

Output

```

3

```

Expected

```

3

```